UNIVERSIDAD TECNOLOGICA NACIONAL

Trabajo Práctico Integrador

Hardening de un servidor Linux: detección y mitigación de accesos no autorizados por SSH

Alumno

Gustavo Enrique Valenzuela

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación I

Coordinador

Prof. Oscar Alberto Londero

Profesores

Prof. Martín León Aristiaran

Tutor

Emmanuel Avellaneda

Link al video: https://www.youtube.com/watch?v=5O4gtPTf2II

Repositorio / carpeta digital: https://github.com/valenzuelagustavo/tpi-hardening-ssh

07 de noviembre de 2026

Tabla de contenido

1. Introducción 3
2. Marco teórico 4
2.1 Amenazas y superficie de ataque 4
2.2 SSH y métodos de autenticación 4
2.3 Mínimo privilegio y control de acceso 4
2.4 Firewall 4
2.5 Registros y respuesta automática 5
2.6 Gestión de actualizaciones 5
2.7 Defensa en profundidad 5
3. Caso práctico 6
3.1 Entorno de trabajo 6
3.2 Etapa 1: línea base (estado inicial) 11
3.3 Etapa 2: incidente controlado 13
3.4 Etapa 3: aplicación del hardening 14
3.4.1 Autenticación por clave pública y endurecimiento de SSH 16
3.4.2 Firewall (UFW) 16
3.4.3 Fail2ban 17
3.4.4 Actualizaciones automáticas 20
3.4.5 Comparación antes/después 21
3.5 Automatización: script hardening.sh 22
4. Metodología utilizada 24
5. Resultados obtenidos 25
5.1 Comparación antes/después 25
5.2 Evidencia del incidente 25
5.3 Efecto de las defensas 25
5.4 Problemas encontrados y soluciones 26
5.5 Limitaciones 26
6. Conclusiones 27
7. Nota de transparencia 27
8. Bibliografía 28
9. Anexos 29
Anexo A. Script hardening.sh 30
Anexo B. Comandos de verificación utilizados 32
Anexo C. Enlaces y entregables 32

1. Introducción

Este trabajo analiza qué tan expuesto queda un servidor Linux con su configuración por defecto frente a intentos de acceso no autorizados por SSH, y cómo se reduce esa exposición con medidas concretas de endurecimiento (*hardening*). El estudio se realizó sobre una máquina virtual con Ubuntu Server 24.04.5 LTS, aislada en una red de laboratorio.

El tema se eligió porque SSH es el servicio de administración remota más usado en servidores Linux y, por lo tanto, uno de los más atacados. Para un técnico en programación, comprender cómo se autentica un usuario, qué registra el sistema operativo y cómo se limita el acceso es parte de desplegar aplicaciones de forma responsable: un programa bien escrito no es seguro si el servidor que lo aloja está mal configurado. Una de las motivaciones personales que lleva a elegir el tema en cuestión es la posibilidad de ver de cerca como la seguridad nunca es total. Una sola rama puede romperse, pero una gran cantidad de ellas son muy fuertes. Así mismo, mientras más recaudos y medidas se tomen, más seguro será el sistema.

Los objetivos del trabajo son:

1. Medir y documentar el estado de seguridad del servidor antes de modificarlo (línea base).
2. Reproducir un acceso no autorizado en un entorno controlado y analizar la evidencia que deja en los registros del sistema.
3. Aplicar autenticación por clave, un firewall, bloqueo automático de IP y actualizaciones automáticas, verificando el valor efectivo de cada medida.
4. Comparar el estado antes y después, y automatizar el proceso con un script reproducible.

Todas las pruebas se hicieron sobre una máquina propia, sin exponerla a internet ni atacar equipos de terceros.

2. Marco teórico

La seguridad de un sistema operativo se apoya en mecanismos que controlan quién accede, a qué recursos y qué queda registrado de esa actividad (Silberschatz, Galvin & Gagne, 2018). Este trabajo se centra en la protección del acceso remoto por SSH.

2.1 Amenazas y superficie de ataque

La *superficie de ataque* es el conjunto de servicios y configuraciones por los que un sistema puede ser atacado. Un servidor con SSH abierto en el puerto 22 es alcanzable por cualquier equipo con conectividad de red, y una de las amenazas más habituales es la *fuerza bruta*: probar muchas combinaciones de usuario y contraseña hasta acertar. Reducir la superficie de ataque (cerrar lo innecesario, limitar quién puede entrar) es el primer principio del endurecimiento.

2.2 SSH y métodos de autenticación

SSH (*Secure Shell*) es un protocolo cliente-servidor que permite administrar un equipo de forma remota con la comunicación cifrada. En Linux lo provee el servicio sshd de OpenSSH, configurable con sshd_config (OpenBSD Project, s. f.). Admite dos métodos de autenticación relevantes:

- **Contraseña:** el secreto viaja (cifrado) hasta el servidor y se compara allí. Es vulnerable a adivinación y a reutilización de contraseñas débiles.
- **Clave pública:** se genera un par de claves. La clave pública se instala en el servidor (authorized_keys) y la privada permanece en el cliente. El cliente demuestra que posee la privada sin enviarla. Una *passphrase* opcional protege la clave privada en el equipo local.

2.3 Mínimo privilegio y control de acceso

El principio de mínimo privilegio indica que cada usuario o proceso debe tener solo los permisos imprescindibles. En SSH se aplica, por ejemplo, impidiendo el acceso directo del usuario root (PermitRootLogin), limitando qué cuentas pueden conectarse (AllowUsers) y delegando tareas administrativas mediante sudo, que además queda registrado.

2.4 Firewall

Un firewall filtra el tráfico de red según reglas. Con una política *deny* por defecto, todo lo entrante se descarta salvo lo permitido explícitamente. UFW (*Uncomplicated Firewall*) es una interfaz simplificada para administrar el filtrado de paquetes del núcleo de Linux.

2.5 Registros y respuesta automática

El sistema registra los eventos de autenticación en el *journal* de systemd y en /var/log/auth.log. Leer esos registros permite detectar incidentes, pero hacerlo a mano no escala. Herramientas como **fail2ban** automatizan la respuesta: una *jail* observa un servicio, un *filtro* reconoce líneas de fallo mediante expresiones regulares y una *acción* bloquea la IP de origen. Se controla con tres parámetros: maxretry (fallos tolerados), findtime (ventana en que se cuentan) y bantime (duración del bloqueo).

2.6 Gestión de actualizaciones

Muchas intrusiones aprovechan vulnerabilidades ya corregidas. unattended-upgrades instala parches automáticamente según los orígenes habilitados; en Ubuntu, por defecto, solo los de seguridad.

2.7 Defensa en profundidad

Ninguna medida alcanza por sí sola. La *defensa en profundidad* combina capas independientes, de modo que si una falla, otra contiene el daño. La siguiente tabla resume las capas aplicadas en este trabajo.

Capa	Medida	Herramienta	Riesgo que mitiga
Autenticación	Solo clave pública	sshd	Adivinar o robar contraseñas
Privilegios	Sin login de root, AllowUsers	sshd	Cuenta privilegiada expuesta
Red	Deny por defecto, solo SSH	UFW	Servicios innecesarios expuestos
Detección y respuesta	Bloqueo tras 3 fallos	fail2ban	Intentos reiterados de acceso
Mantenimiento	Parches de seguridad automáticos	unattended-upgrades	Vulnerabilidades conocidas

3. Caso práctico

El caso práctico se desarrolló en tres etapas sobre una misma máquina virtual: (1) relevamiento de la línea base, (2) simulación controlada de un acceso no autorizado y (3) aplicación de medidas de hardening con verificación posterior. Cada etapa terminó con una instantánea (*snapshot*) de VirtualBox, lo que permite volver a un estado conocido y repetir la demostración.

3.1 Entorno de trabajo

Elemento	**Detalle**
Equipo anfitrión	Windows 11, 32 GB de RAM, VirtualBox 7.x
Servidor	Ubuntu Server 24.04.5 LTS (VM ubuntuserver, 4096 MB de RAM, 2 CPU)
Red	Adaptador 1: NAT (salida a Internet). Adaptador 2: solo anfitrión (red 192.168.56.0/24), usado para el acceso SSH desde Windows
Usuario administrador	admin1
Cliente de ataque/pruebas	PowerShell de Windows (OpenSSH cliente), IP 192.168.56.1

La VM se creó con la imagen oficial de Ubuntu Server (Figura 1) y se le asignaron 4096 MB de memoria y 2 procesadores (Figura 2). Se configuraron dos adaptadores de red: NAT (Figura 3) y solo anfitrión (Figura 4), de modo que el servidor tuviera Internet para instalar paquetes y, a la vez, fuera accesible desde Windows sin exponerlo a la red local.

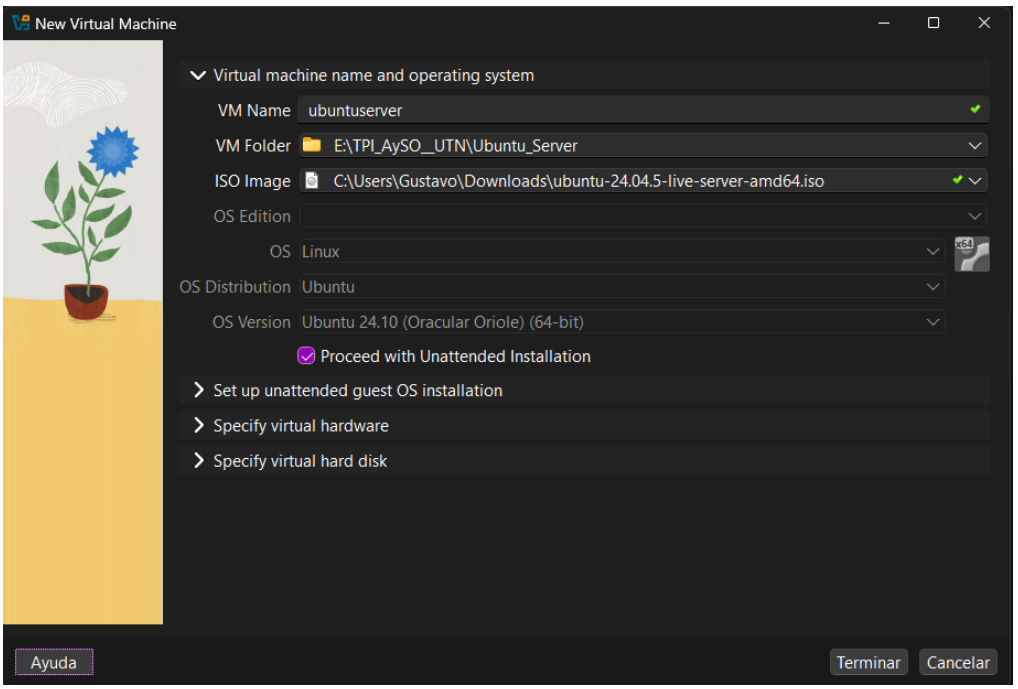


Figura 1 – Asistente de VirtualBox: nombre de la VM e imagen ISO de Ubuntu Server 24.04.5

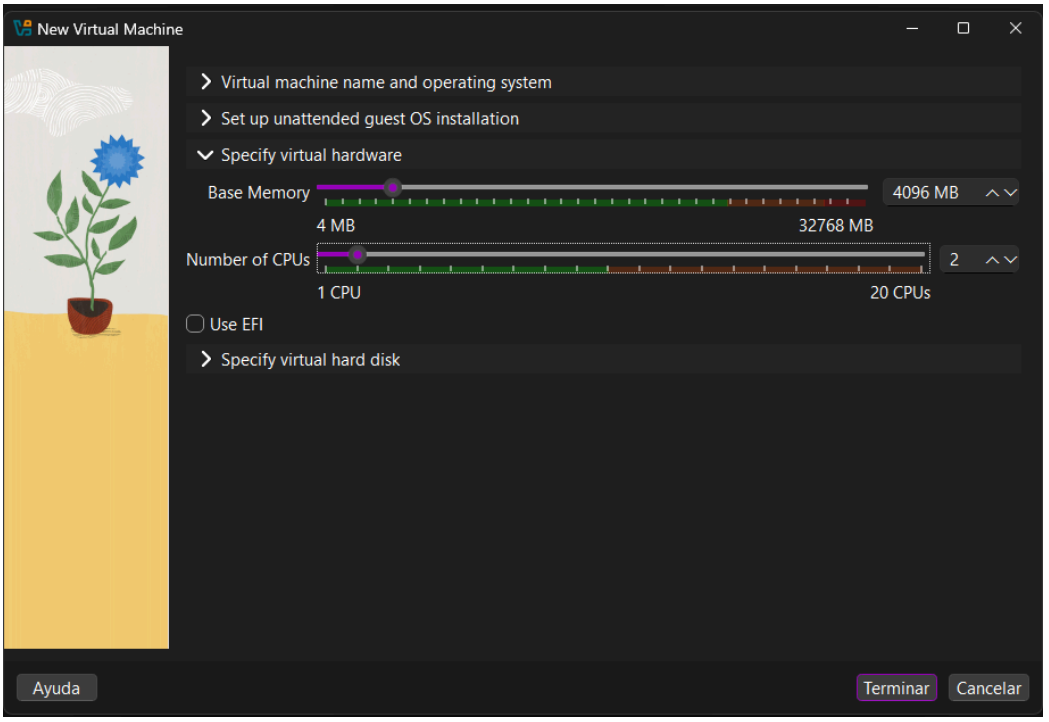


Figura 2 – Hardware asignado a la VM: 4096 MB de RAM y 2 CPU

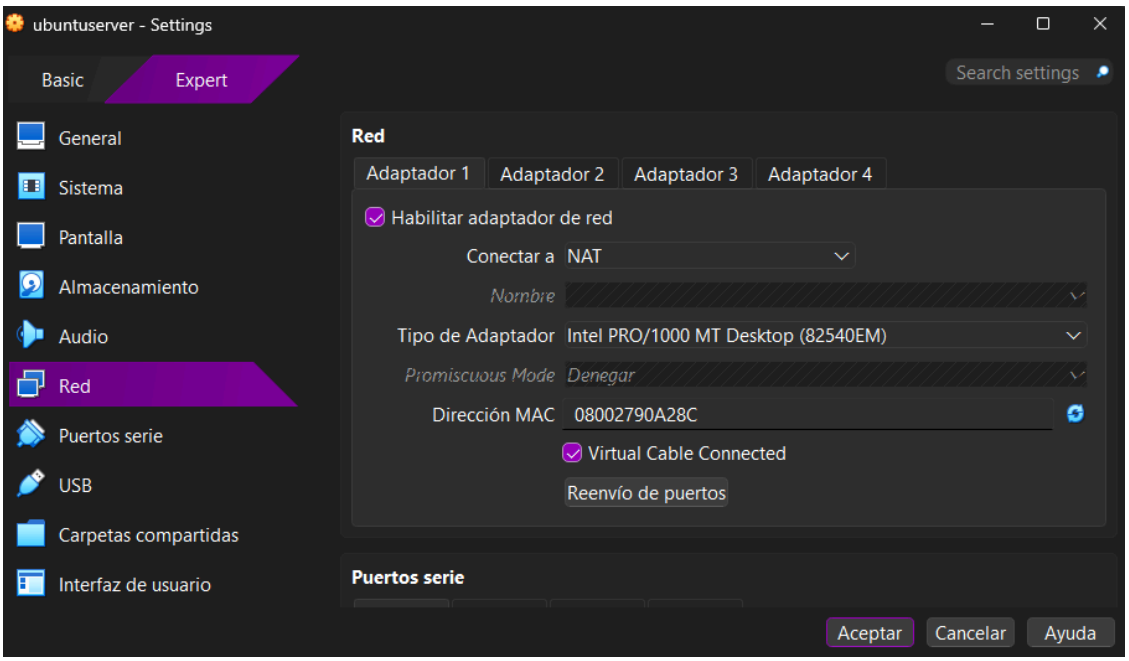


Figura 3 – Adaptador 1 en modo NAT

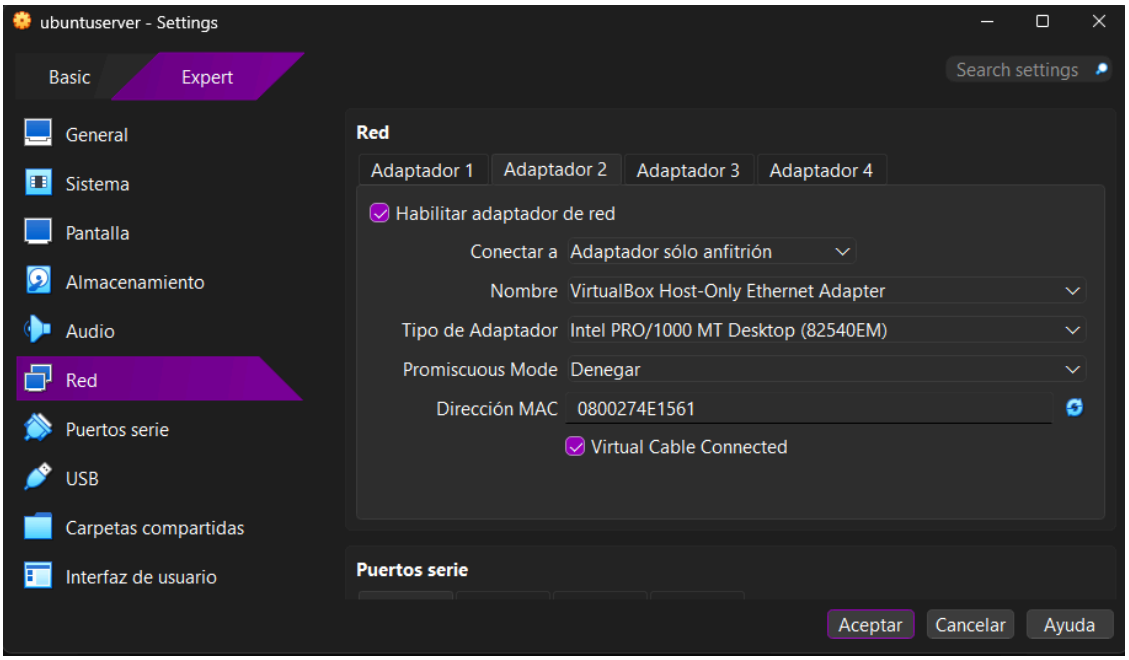


Figura 4 – Adaptador 2 en modo solo anfitrión (Host-Only)

Antes de instalar se verificó la integridad de la imagen: se comparó el SHA256 calculado en Windows con el publicado en el sitio oficial de Ubuntu (Figuras 5 y 6).

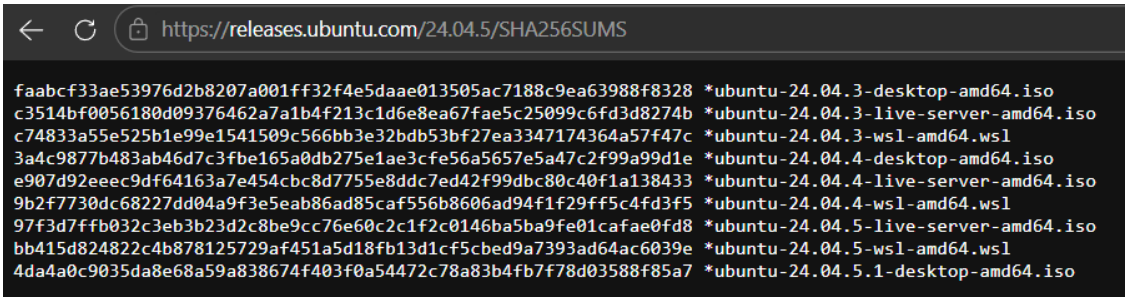


Figura 5 – Lista SHA256SUMS publicada por Ubuntu para la versión 24.04.5

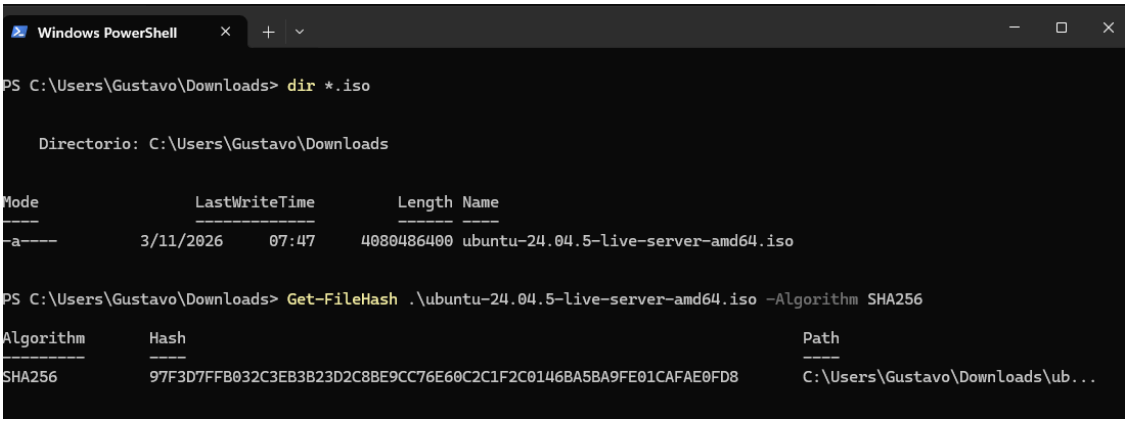


Figura 6 – Hash SHA256 de la ISO descargada, calculado con Get-FileHash

Tras la instalación, la interfaz solo anfitrión quedó con la dirección 192.168.56.102 (Figura 7). Como la instalación automática no incluyó el servidor SSH, se instaló el paquete openssh-server y se comprobó que el servicio quedará activo y escuchando en el puerto 22 (Figura 8).

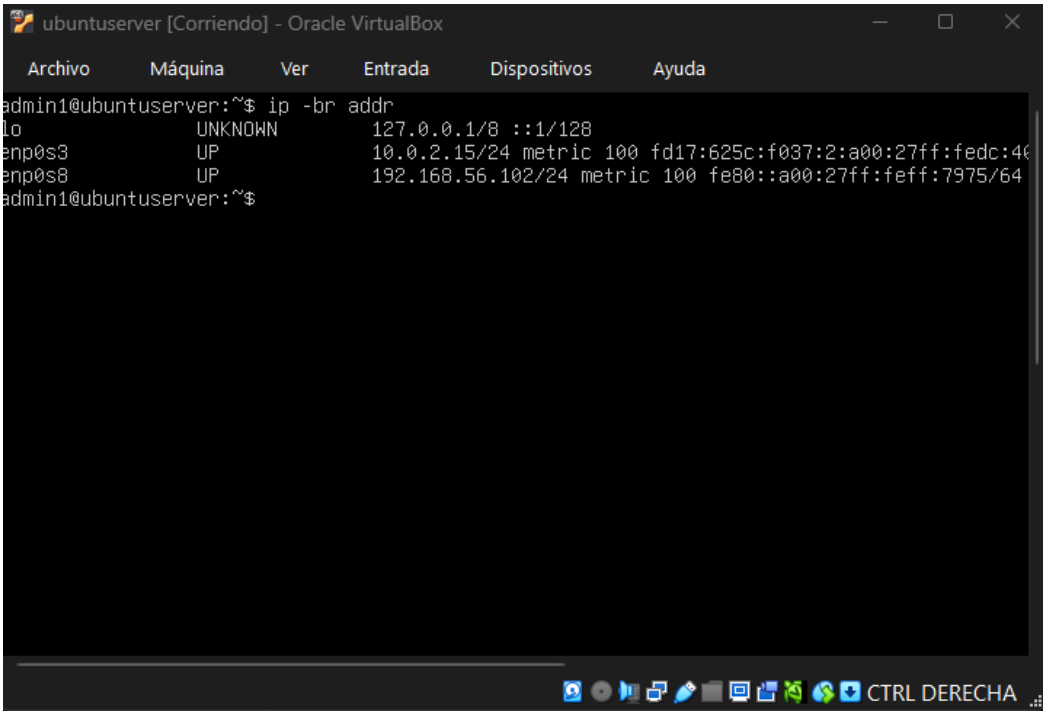


Figura 7 – Direcciones de red de la VM (ip -br addr)

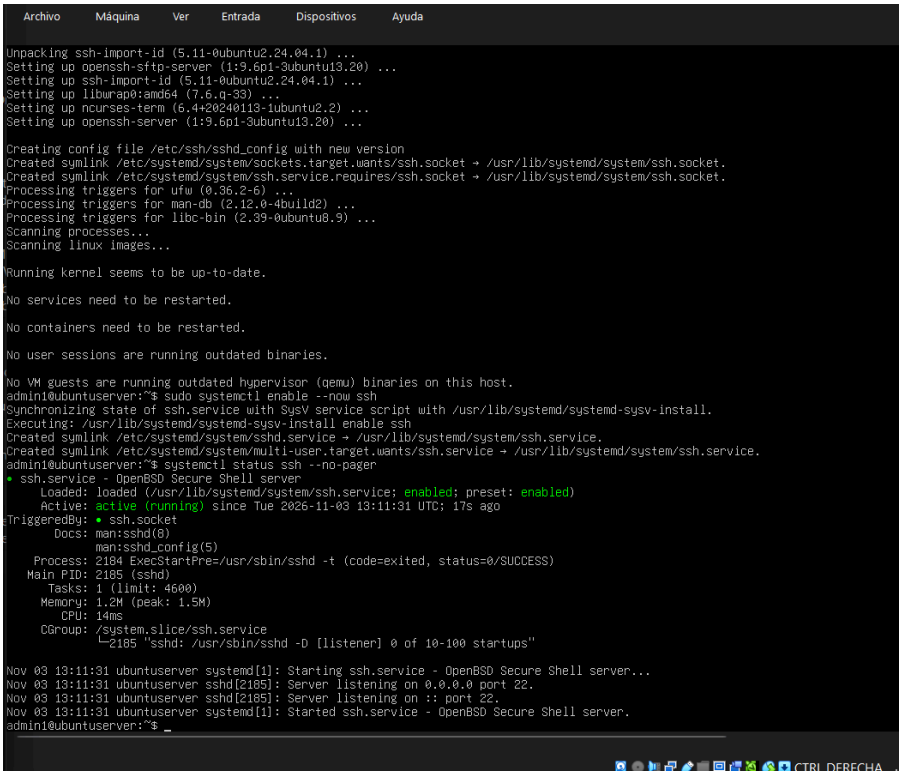


Figura 8 – Instalación de openssh-server y verificación del servicio ssh

3.2 Etapa 1: línea base (estado inicial)

Antes de modificar nada se documentó el estado inicial del servidor, guardando la evidencia en archivos de texto (~/evidencia/antes/): puertos en escucha, valores efectivos de SSH con sshd -T, estado del firewall, usuarios del sistema y actualizaciones pendientes (Figuras 9 y 10).

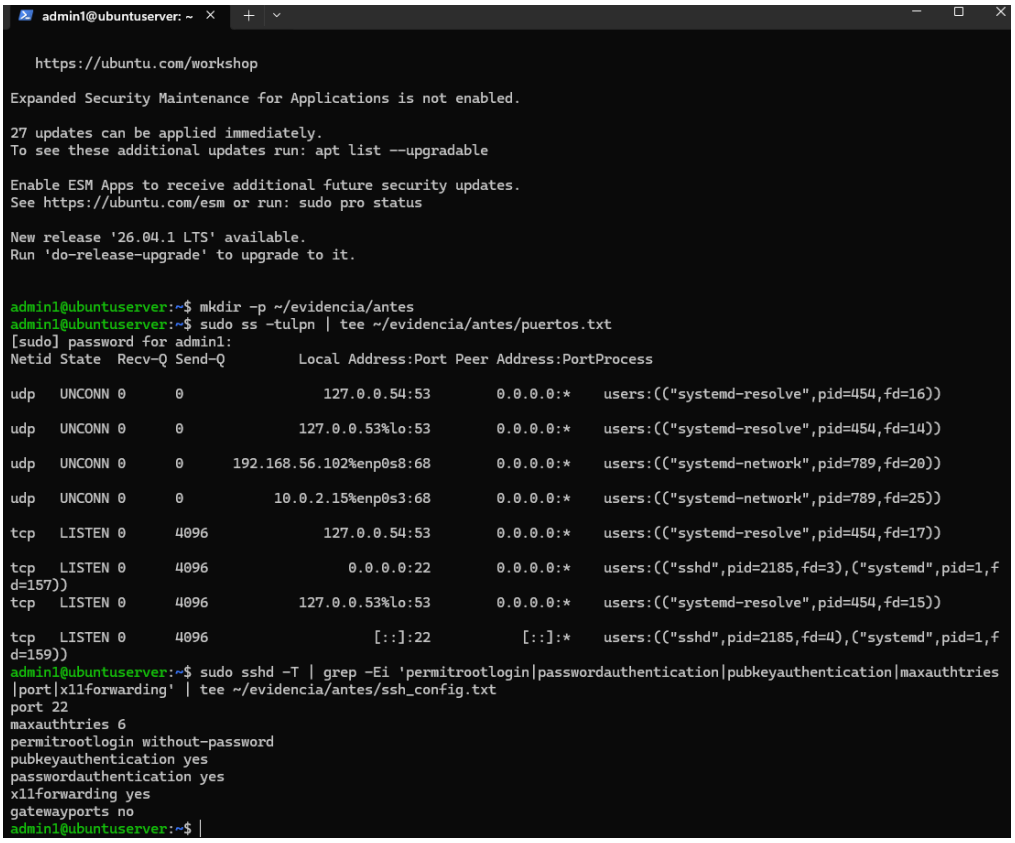


Figura 9 – Mensaje de bienvenida del servidor (27 actualizaciones pendientes) y captura de los puertos en escucha

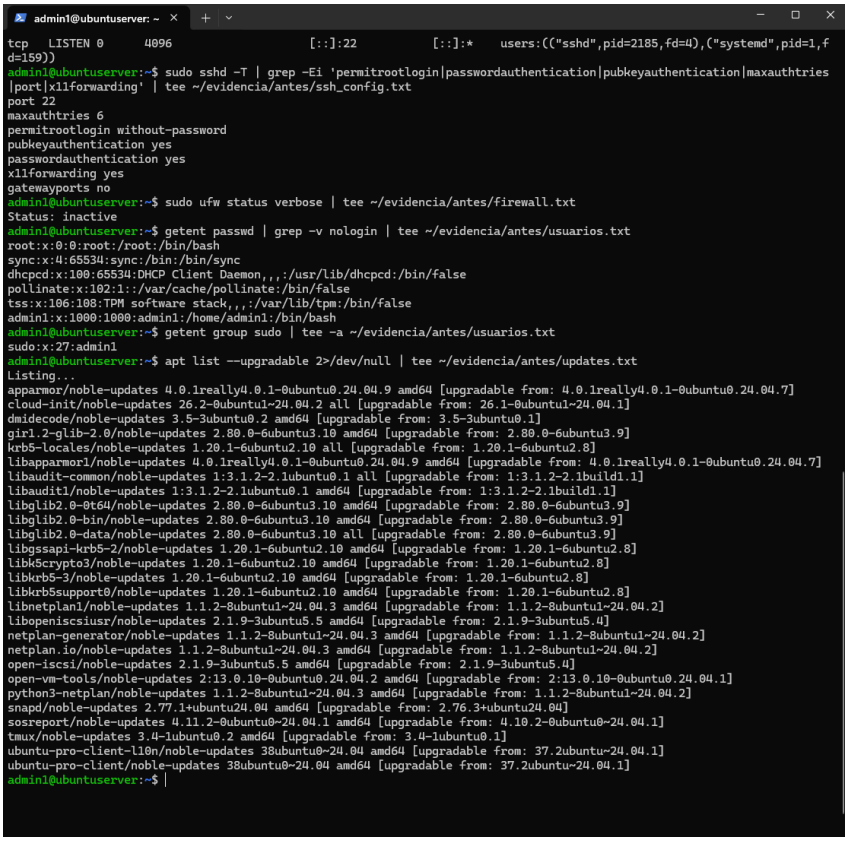


Figura 10 – Evidencia de la línea base: configuración efectiva de SSH, firewall inactivo y paquetes actualizables

Los hallazgos principales de la línea base fueron:

- SSH escuchando en 0.0.0.0:22 y [::]:22.
- PermitRootLogin without-password, PasswordAuthentication yes, MaxAuthTries 6 y X11Forwarding yes.
- Firewall UFW inactivo.
- 27 actualizaciones pendientes.
- El directorio /etc/ssh/sshd_config.d/ vacío y sin configuración propia (Figura 11).

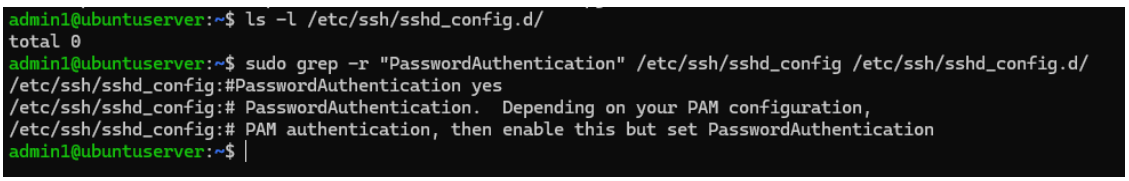


Figura 11 – sshd_config.d vacío: la configuración de SSH es la de fábrica

Al finalizar la etapa se tomó la instantánea 01-linea-base.

3.3 Etapa 2: incidente controlado

Para observar cómo se ve un intento de acceso no autorizado, se simularon intentos fallidos desde Windows contra la propia VM (un entorno aislado de laboratorio, sin afectar a terceros). Se probó primero con un usuario inexistente (prueba) y luego con el usuario real admin1 y una contraseña incorrecta (Figura 12).

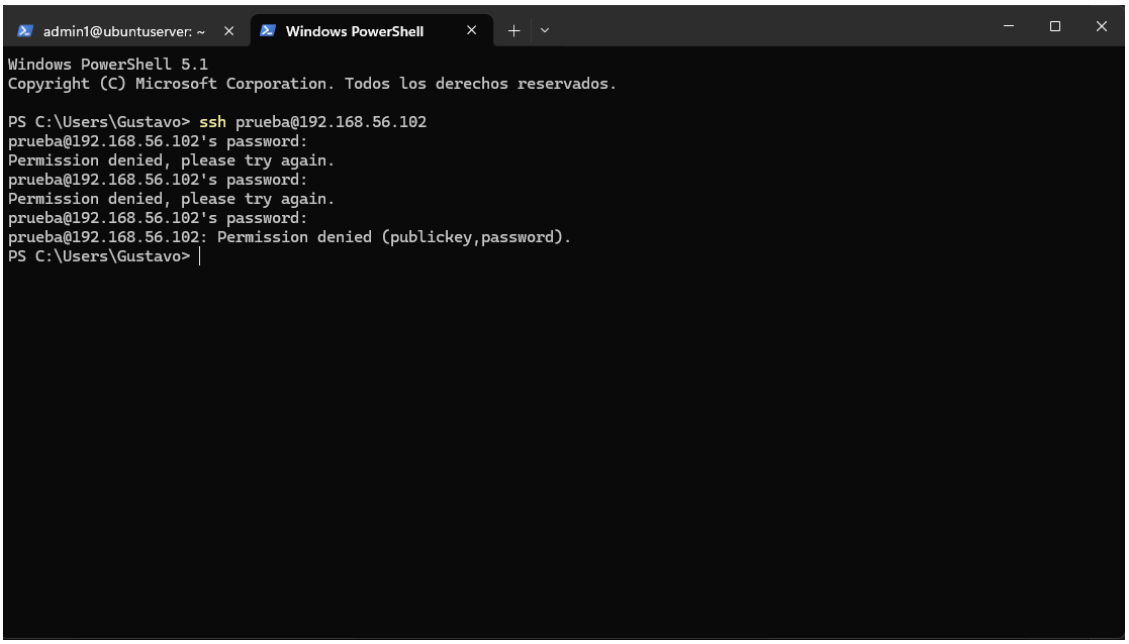


Figura 12 – Intentos fallidos de autenticación desde Windows con el usuario prueba

El servidor registró cada intento en el log de autenticación. En el diario de sshd se distinguen Invalid user prueba (usuario inexistente) y Failed password for admin1 (usuario válido con clave incorrecta), todos originados en 192.168.56.1 (Figura 13). Al contar los eventos se obtuvieron 6 intentos fallidos desde una única IP (Figura 14). Se tomó la instantánea 02-incidente-sin-defensa.

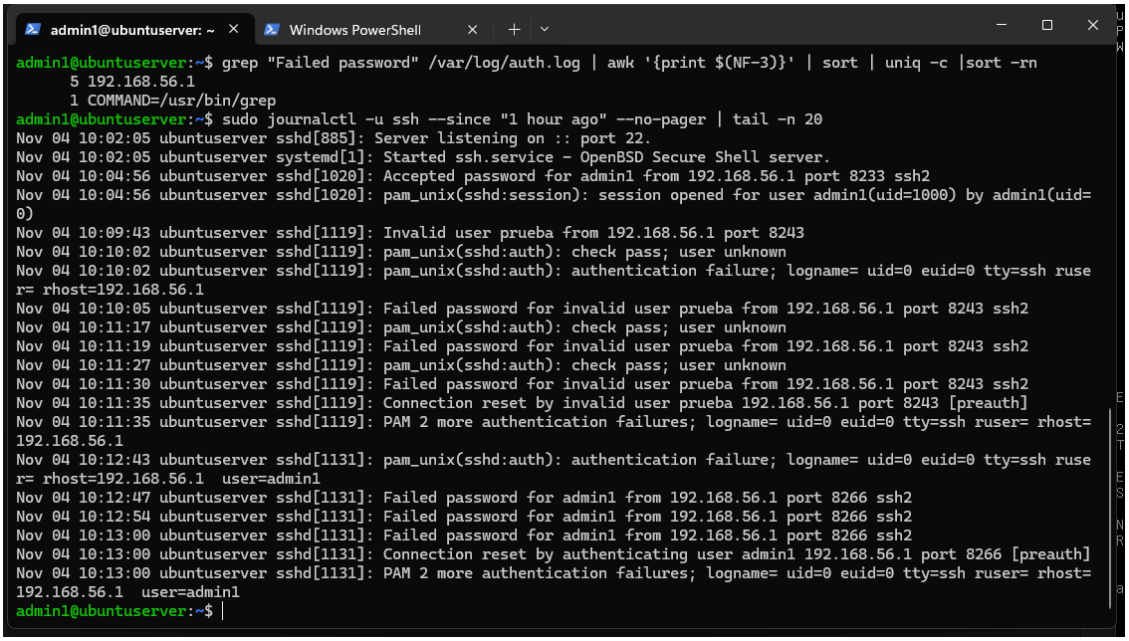


Figura 13 – Registro de los intentos fallidos en el journal de sshd

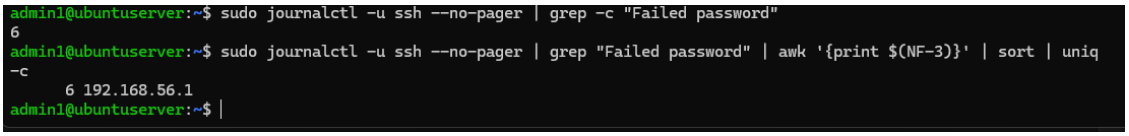


Figura 14 – Conteo de intentos fallidos (6) agrupados por IP de origen

La conclusión de esta etapa es que, sin defensas, el servidor permite seguir intentando contraseñas indefinidamente (con un límite de 6 por conexión) y no existe ninguna reacción automática ante la repetición de fallos.

3.4 Etapa 3: aplicación del hardening

3.4.1 Autenticación por clave pública y endurecimiento de SSH

Se generó un par de claves ed25519 protegido con *passphrase* en Windows (Figura 15), se copió la clave pública a ~/.ssh/authorized_keys del usuario admin1 y se comprobó el acceso por clave antes de desactivar las contraseñas (Figura 16).

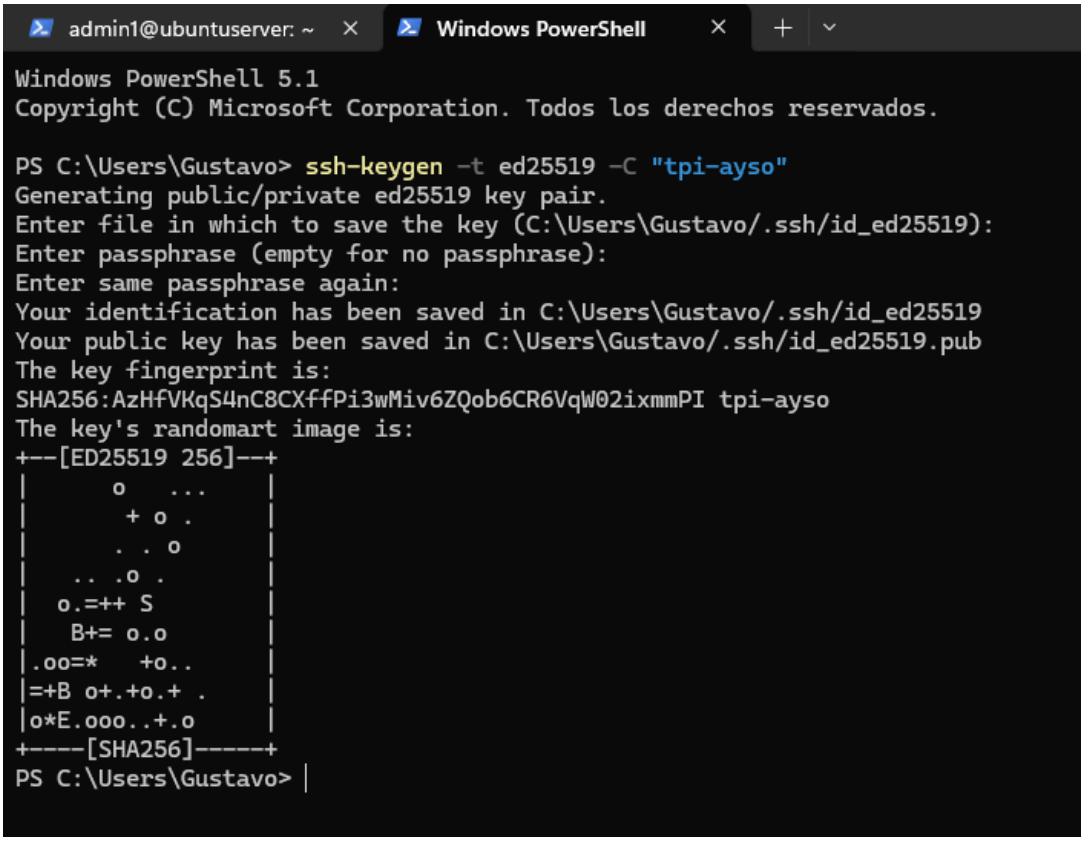


Figura 15 – Generación del par de claves ed25519 con ssh-keygen

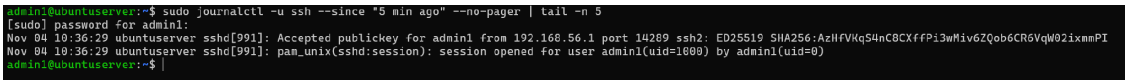


Figura 16 – El servidor registra Accepted publickey for admin1

Luego se creó el archivo /etc/ssh/sshd_config.d/00-hardening.conf (Figura 17) con las siguientes directivas: PermitRootLogin no, PasswordAuthentication no, PubkeyAuthentication yes, MaxAuthTries 3, LoginGraceTime 30, X11Forwarding no y AllowUsers admin1. Se usó el prefijo 00- porque, en los archivos de sshd_config.d, el primer valor leído para cada directiva es el que prevalece.

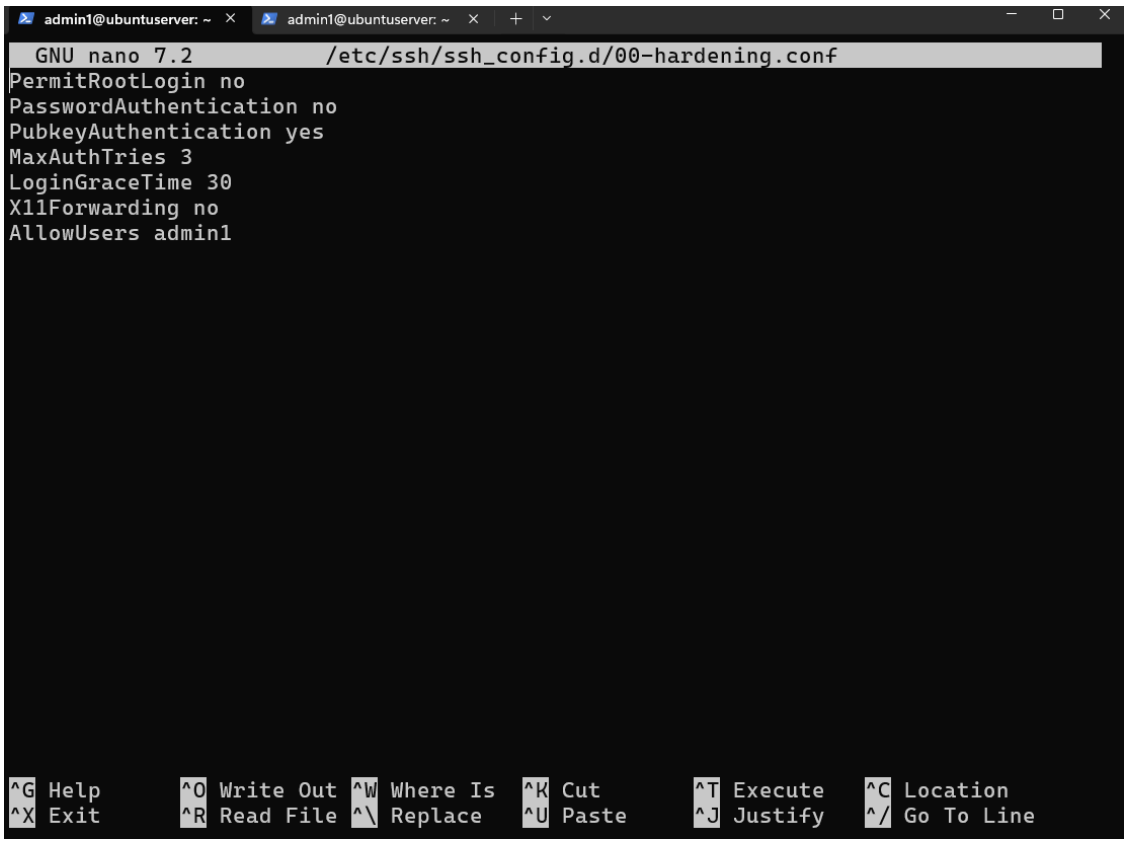


Figura 17 – Archivo 00-hardening.conf editado con nano

Error encontrado y corregido. En un primer intento el archivo se creó por error en /etc/ssh/ssh_config.d/ (directorio de configuración del *cliente*) y el cliente SSH respondió con Bad configuration option (Figura 18). La corrección fue moverlo a /etc/ssh/sshd_config.d/ (directorio del *servidor*). Es un error fácil de cometer por la similitud de nombres, y es útil documentarlo.

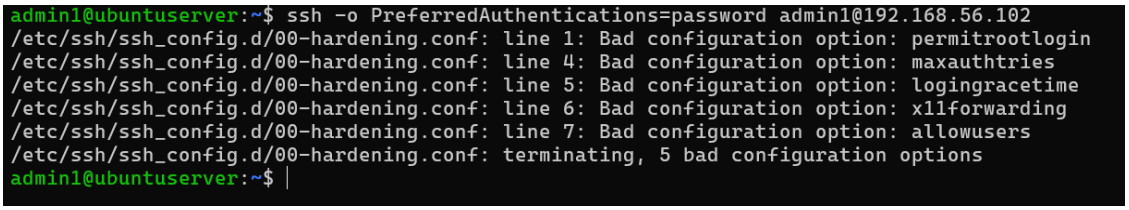


Figura 18 – Error Bad configuration option por ubicar el archivo en el directorio del cliente

3.4.2 Firewall (UFW)

Se configuró UFW con política de entrada denegada por defecto y salida permitida, habilitando únicamente el perfil OpenSSH **antes** de activar el firewall, para no cortar la

sesión en curso (Figura 19). La misma figura muestra los valores efectivos de sshd -T ya con el hardening aplicado.

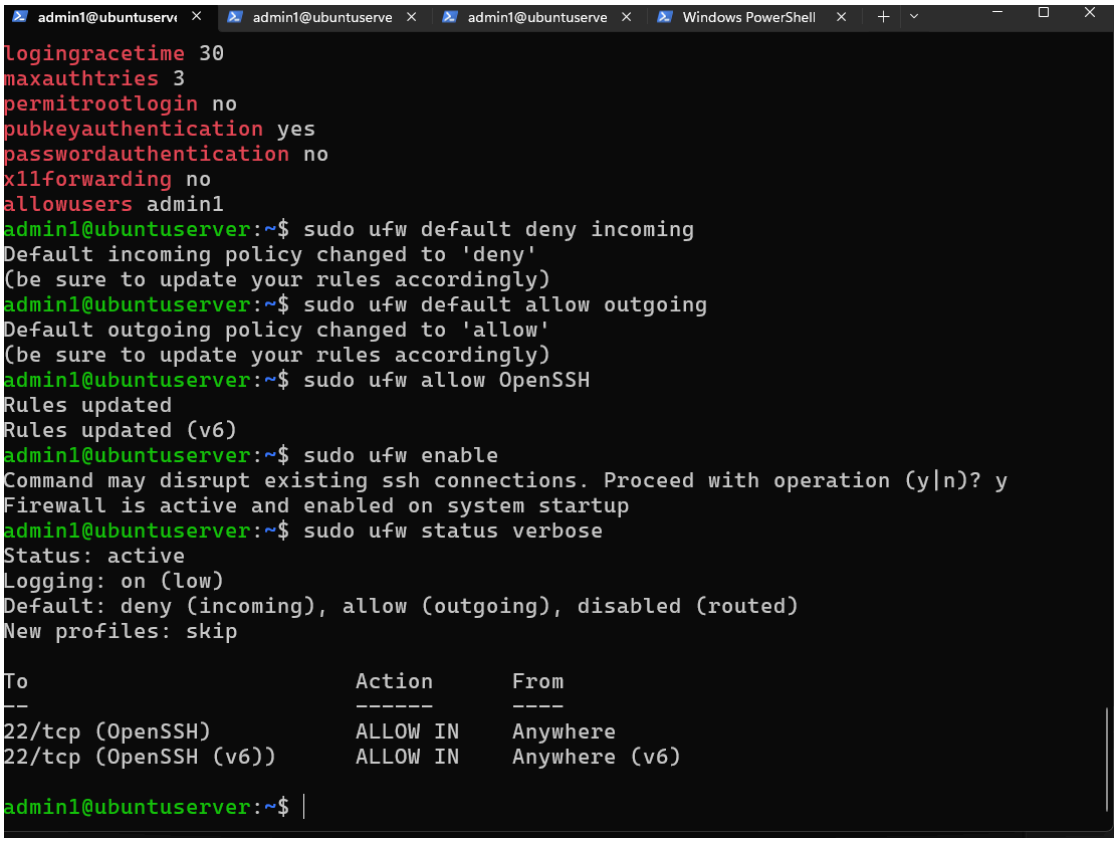


Figura 19 – Valores efectivos de SSH tras el hardening y configuración de UFW

3.4.3 Fail2ban

Se instaló fail2ban y se creó /etc/fail2ban/jail.local con maxretry = 3, findtime = 10m, bantime = 10m, banaction = ufw y la cárcel [sshd] habilitada con backend systemd.

Detalle importante. Al instalarse, el paquete inicia el servicio con valores por defecto, antes de que exista el jail.local. Por eso, inmediatamente después de crear el archivo, fail2ban-client seguía mostrando maxretry 5 y la acción nftables (Figura 20). Fue necesario reiniciar el servicio (systemctl restart fail2ban) para que tomara la configuración propia: luego del reinicio, maxretry pasó a 3 y la acción a ufw (Figura 22).

```
admin1@ubuntuserver:~$ sudo cat /etc/fail2ban/jail.local
[DEFAULT]
bantime  = 10m
findtime = 10m
maxretry = 3
banaction = ufw

[sshd]
enabled = true
backend = systemd
admin1@ubuntuserver:~$ sudo fail2ban-client get sshd maxretry
5
admin1@ubuntuserver:~$ sudo fail2ban-client get sshd actions
The jail sshd has the following actions:
nftables
admin1@ubuntuserver:~$
```

Figura 20 – Configuración escrita en jail.local, pero fail2ban aún con valores por defecto (maxretry 5, nftables)

```
admin1@ubuntuserver:~$ sudo grep -Ei "found|ban|error|warn" /var/log/fail2ban.log | tail -n 30
2026-11-04 10:51:31,360 fail2ban.server         [1851]: INFO    --------------------------------------------------
2026-11-04 10:51:31,360 fail2ban.server         [1851]: INFO    Starting Fail2ban v1.0.2
2026-11-04 10:51:31,361 fail2ban.observer       [1851]: INFO    Observer start...
2026-11-04 10:51:31,385 fail2ban.database       [1851]: INFO    Connected to fail2ban persistent database '/var/lib/fail2ban/fail2ban.sqlite3'
2026-11-04 10:51:31,387 fail2ban.database       [1851]: WARNING New database created. Version '4'
2026-11-04 10:51:31,387 fail2ban.jail           [1851]: INFO    Creating new jail 'sshd'
2026-11-04 10:51:32,005 fail2ban.jail           [1851]: INFO    Jail 'sshd' uses systemd {}
2026-11-04 10:51:32,006 fail2ban.jail           [1851]: INFO    Initiated 'systemd' backend
2026-11-04 10:51:32,006 fail2ban.filter         [1851]: INFO      maxLines: 1
2026-11-04 10:51:32,047 fail2ban.filtersystemd  [1851]: INFO    [sshd] Added journal match for: '_SYSTEMD_UNIT=sshd.service + _COMM=sshd'
2026-11-04 10:51:32,047 fail2ban.filter         [1851]: INFO      maxRetry: 5
2026-11-04 10:51:32,047 fail2ban.filter         [1851]: INFO      findtime: 600
2026-11-04 10:51:32,047 fail2ban.actions        [1851]: INFO      banTime: 600
2026-11-04 10:51:32,047 fail2ban.filter         [1851]: INFO      encoding: UTF-8
2026-11-04 10:51:32,049 fail2ban.jail           [1851]: INFO    Jail 'sshd' started
2026-11-04 10:51:32,060 fail2ban.filtersystemd  [1851]: INFO    [sshd] Jail is in operation now (process new journal entries)
2026-11-04 10:54:03,696 fail2ban.filter         [1851]: INFO    [sshd] Found 192.168.56.1 - 2026-11-04 10:54:03
2026-11-04 10:54:09,542 fail2ban.filter         [1851]: INFO    [sshd] Found 192.168.56.1 - 2026-11-04 10:54:09
2026-11-04 10:54:12,553 fail2ban.filter         [1851]: INFO    [sshd] Found 192.168.56.1 - 2026-11-04 10:54:12
2026-11-04 10:57:42,337 fail2ban.filter         [1851]: INFO    [sshd] Found 192.168.56.1 - 2026-11-04 10:57:41
2026-11-04 10:57:42,338 fail2ban.filter         [1851]: INFO    [sshd] Found 192.168.56.1 - 2026-11-04 10:57:42
2026-11-04 10:57:42,916 fail2ban.actions        [1851]: NOTICE  [sshd] Ban 192.168.56.1
2026-11-04 11:07:42,280 fail2ban.actions        [1851]: NOTICE  [sshd] Unban 192.168.56.1
admin1@ubuntuserver:~$
```

Figura 21 – Registro de fail2ban: detección de intentos (Found) y bloqueo (Ban) de 192.168.56.1

```
admin1@ubuntuserver:~$ sudo systemctl restart fail2ban
admin1@ubuntuserver:~$ sudo fail2ban-client get sshd maxretry
3
admin1@ubuntuserver:~$ sudo fail2ban-client get sshd actions
The jail sshd has the following actions:
ufw
admin1@ubuntuserver:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
admin1@ubuntuserver:~$
```

Figura 22 – Tras reiniciar el servicio: maxretry 3, acción ufw y jail activa

Prueba de funcionamiento. Se repitieron los intentos fallidos desde Windows. Al tercer fallo, fail2ban bloqueó la IP 192.168.56.1: en UFW apareció una regla REJECT IN comentada como *by Fail2Ban after 3 attempts against sshd* (Figura 23) y, desde Windows, el servidor pasó a no responder (Connection timed out, Figura 24). Cabe aclarar que se esperaba un Connection refused, pero lo observado fue un *timeout*; en el informe se documenta el comportamiento real. Finalmente, se levantó el bloqueo manualmente con fail2ban-client set sshd unbanip y la regla desapareció (Figura 25).

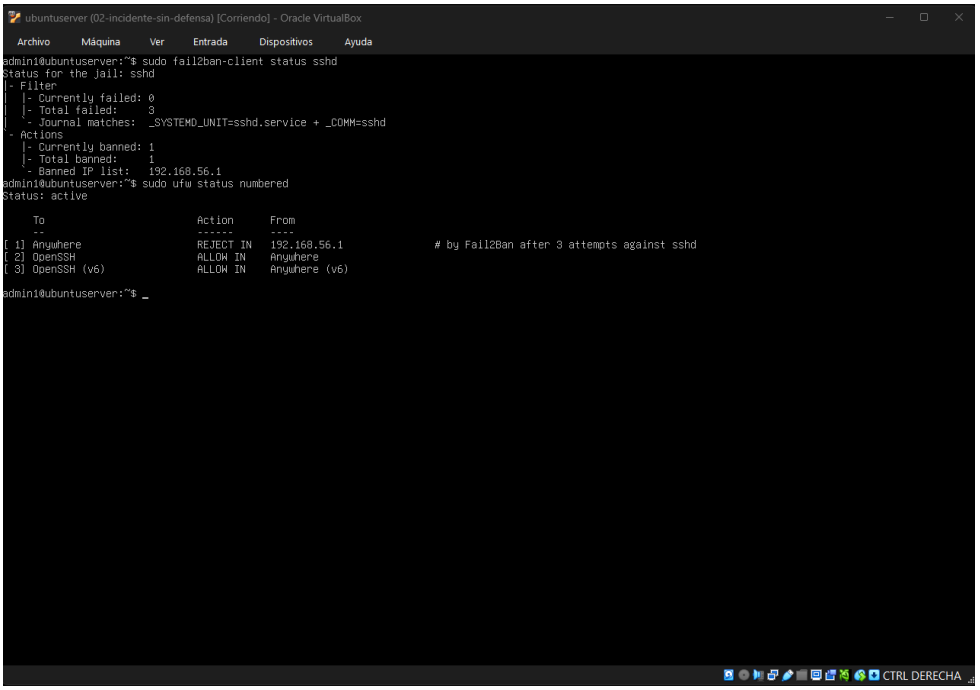


Figura 23 – Regla de bloqueo creada por fail2ban en UFW

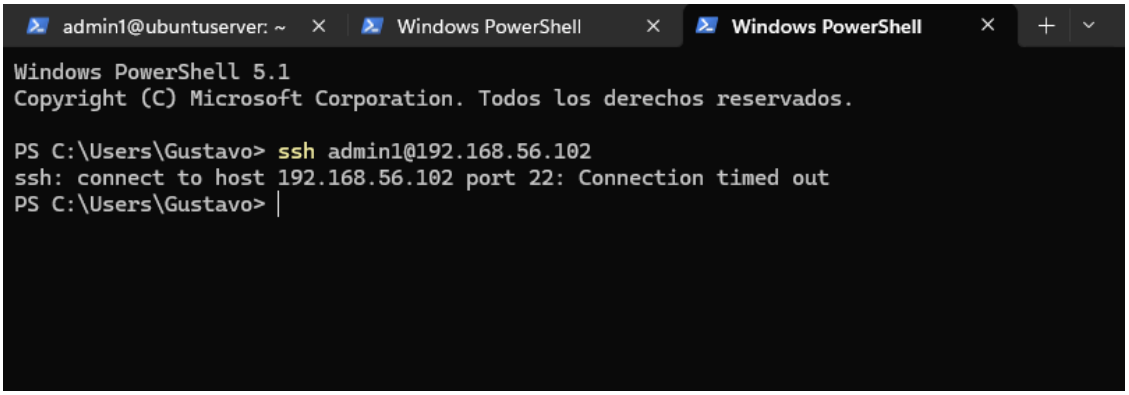


Figura 24 – Desde Windows: Connection timed out mientras la IP está bloqueada

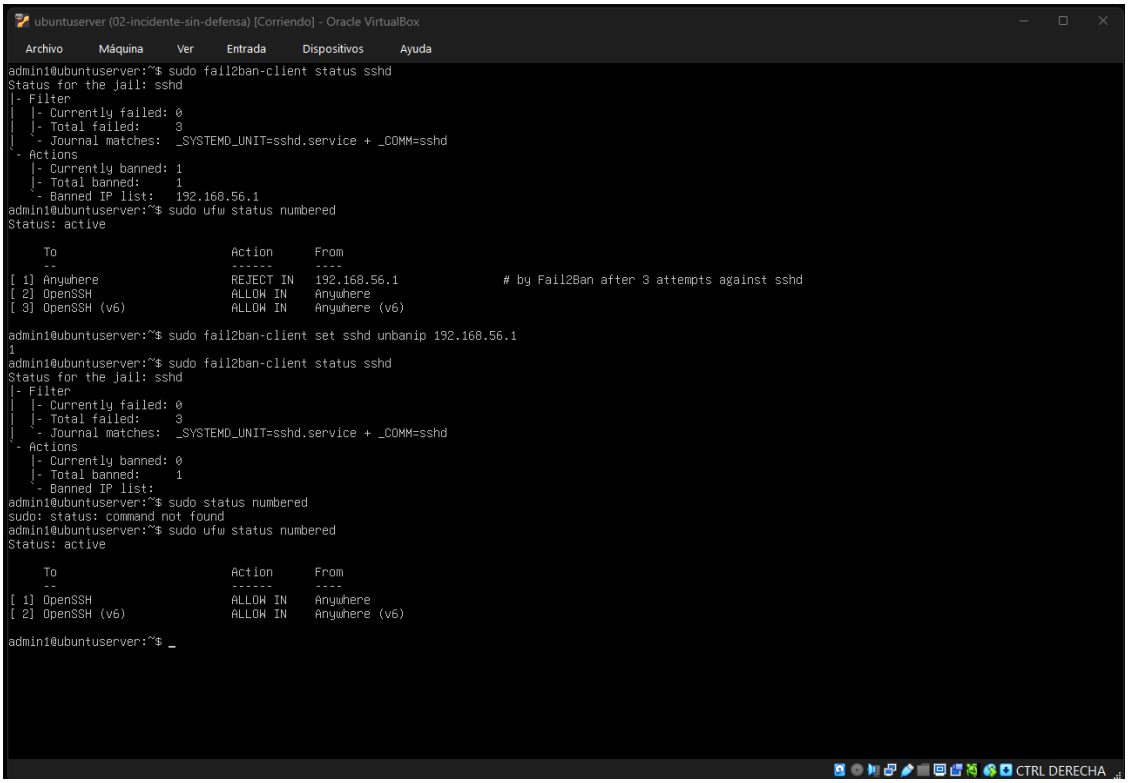


Figura 25 – Desbloqueo manual de la IP y regla eliminada de UFW

3.4.4 Actualizaciones automáticas

Se comprobó que unattended-upgrades ya estaba activo por defecto en Ubuntu Server: los temporizadores apt-daily y apt-daily-upgrade estaban programados y la simulación (--dry-run) funcionaba. Su configuración solo habilita los orígenes de seguridad (-security y ESM); el origen -updates está comentado (Figura 26). Por lo tanto, la medida no fue "activar" las actualizaciones automáticas sino **verificar** su estado y **aplicar** las 27 actualizaciones pendientes, tras lo cual apt list --upgradable quedó vacío (Figura 27).

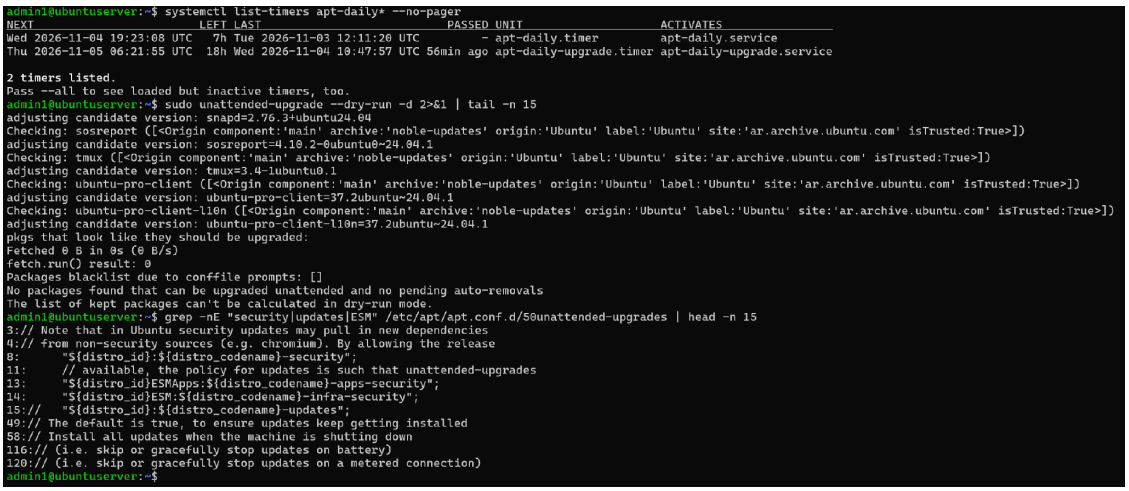


Figura 26 – Temporizadores de apt, simulación de unattended-upgrade y orígenes habilitados

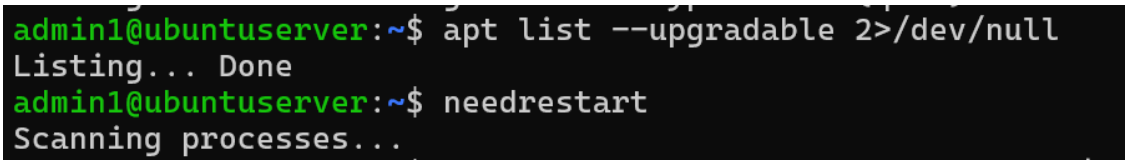


Figura 27 – Sin paquetes pendientes de actualización tras aplicar las actualizaciones

3.4.5 Comparación antes/después

Con la evidencia guardada en ~/evidencia/antes y ~/evidencia/despues se comparó la configuración efectiva de SSH con diff (Figura 28). Se tomó la instantánea 03-hardening-aplicado (Figura 29).

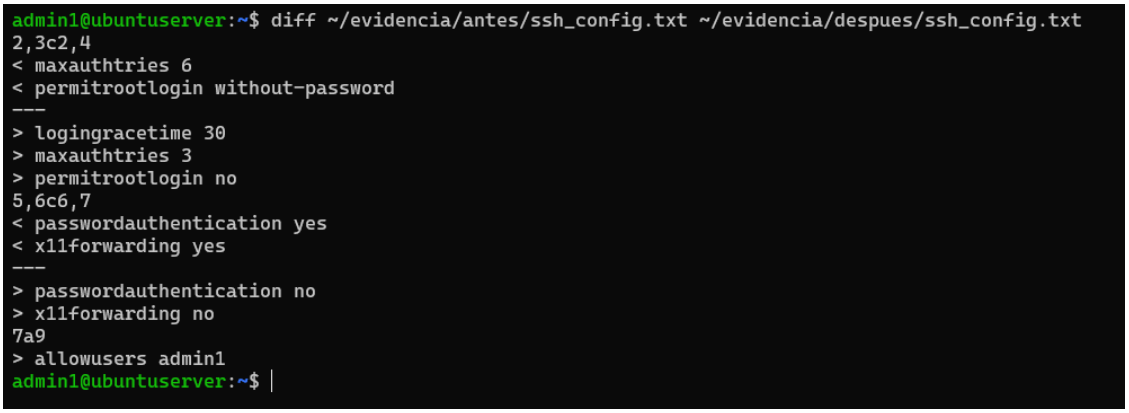


Figura 28 – diff de la configuración efectiva de SSH, antes y después

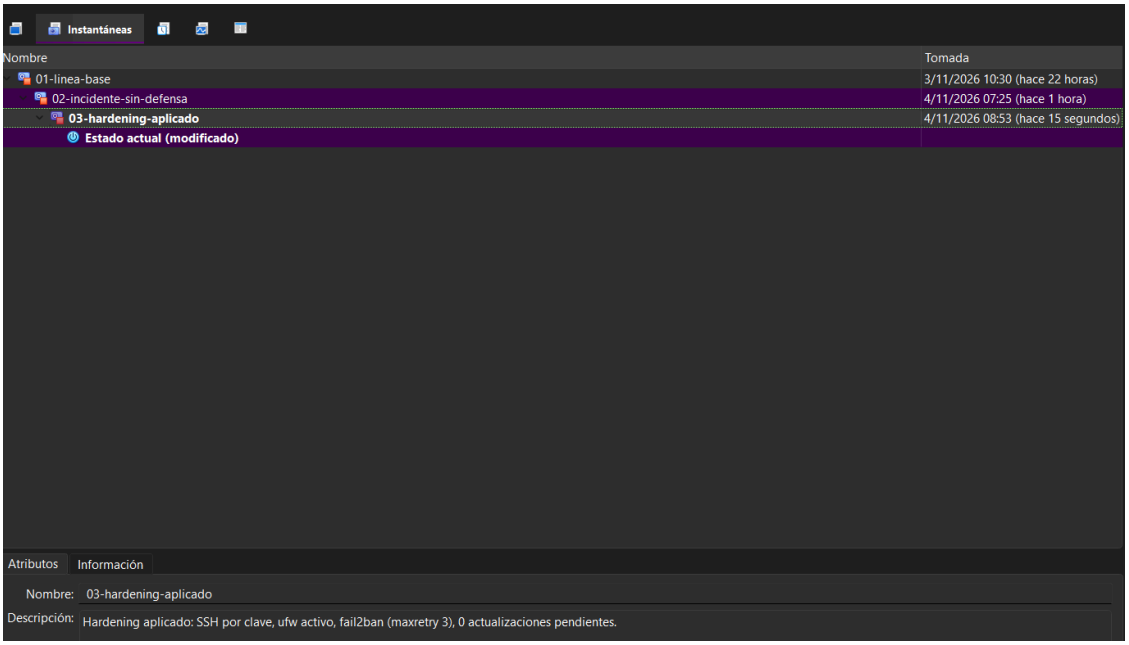


Figura 29 – Instantáneas de la VM: 01-linea-base, 02-incidente-sin-defensa y 03-hardening-aplicado

3.5 Automatización: script hardening.sh

Para que el procedimiento sea repetible, todas las medidas se reunieron en un script de Bash (hardening.sh, ver Anexo A). El script: exige ejecutarse como root; verifica que el usuario administrador tenga una clave pública cargada (para no dejar al administrador sin acceso); escribe 00-hardening.conf y valida la sintaxis con sshd -t (revirtiendo el cambio si es inválida) antes de reiniciar el servicio; configura UFW; escribe jail.local y reinicia fail2ban; y verifica las actualizaciones automáticas, imprimiendo al final el estado efectivo de cada componente (Figura 30).

Para validarlo, se restauró la instantánea 01-linea-base, se cargó la clave pública, se ejecutó el script y se comprobó desde Windows que (a) la contraseña es rechazada con Permission denied (publickey) y (b) el acceso con clave funciona y el servidor informa 0 actualizaciones pendientes (Figura 31).

```
admin1@ubuntuserver:~$ mkdir -p ~/.ssh && chmod 700 ~/.ssh
admin1@ubuntuserver:~$ echo 'ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICAtSVeMO8xCyvybksVWHoxzGmwIO97OGSoyXLXkBd8K tpi-ayso' >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys
admin1@ubuntuserver:~$ sudo bash ~/hardening.sh
[sudo] password for admin1:
Sorry, try again.
[sudo] password for admin1:

[+] Configurando SSH (/etc/ssh/sshd_config.d/00-hardening.conf)

[+] Configurando firewall (UFW)
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Skipping adding existing rule
Skipping adding existing rule (v6)
Firewall is active and enabled on system startup

[+] Configurando fail2ban
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban

[+] Verificando actualizaciones automaticas

[+] Verificacion
--- sshd -T ---
logingracetime 30
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
x11forwarding no
allowusers admin1
--- ufw ---
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)

--- fail2ban (valores efectivos) ---
maxretry: 3
The jail sshd has the following actions:
ufw
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:

[+] Listo. Probar en OTRA terminal que el acceso por clave funciona antes de cerrar esta sesion.
```

Figura 30 – Ejecución de hardening.sh sobre la VM restaurada a la línea base

```
admin1@ubuntuserver: ~ | Windows PowerShell | admin1@ubuntuserver: ~

Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

PS C:\Users\Gustavo> ssh -o PreferredAuthentications=password admin1@192.168.56.102
admin1@192.168.56.102: Permission denied (publickey).
PS C:\Users\Gustavo> ssh admin1@192.168.56.102
Enter passphrase for key 'C:\Users\Gustavo/.ssh/id_ed25519':
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-147-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Nov  4 12:40:46 PM UTC 2026

  System load:             0.0
  Usage of /:              14.2% of 19.51GB
  Memory usage:            6%
  Swap usage:              0%
  Processes:               117
  Users logged in:         1
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fedc:40f0

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '26.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Wed Nov  4 12:30:01 2026 from 192.168.56.1
admin1@ubuntuserver:~$
```

Figura 31 – Pruebas finales: contraseña rechazada y acceso por clave exitoso

4. Metodología utilizada

El trabajo siguió un enfoque experimental de tipo *antes/después*: se mide el estado inicial, se provoca un evento de seguridad, se aplican las medidas y se vuelve a medir. Las etapas fueron:

1. **Preparación del entorno.** Creación de la VM en VirtualBox, verificación de la ISO por hash e instalación de Ubuntu Server 24.04.5 LTS con dos adaptadores de red (NAT y solo anfitrión).
2. **Relevamiento de la línea base.** Registro del estado inicial con herramientas del propio sistema: ss (puertos), sshd -T (configuración efectiva de SSH), ufw status (firewall), getent passwd (usuarios) y apt list --upgradable (actualizaciones). Cada salida se guardó en ~/evidencia/antes/.
3. **Incidente controlado.** Intentos de autenticación fallidos desde Windows hacia la VM (un entorno propio y aislado), analizados luego en auth.log y en journalctl -u ssh.
4. **Aplicación del hardening.** Configuración de SSH por clave, UFW, fail2ban y actualizaciones, siguiendo el principio de menor privilegio y reducción de superficie de ataque (ver Marco teórico).
5. **Verificación.** Se repitió el relevamiento en ~/evidencia/despues/, se comparó con diff y se repitieron los intentos fallidos para comprobar el bloqueo.
6. **Automatización y validación.** El procedimiento se convirtió en un script (hardening.sh) que se probó sobre una VM restaurada a la línea base.

Criterios metodológicos.

- Se comparan siempre los valores *efectivos* (sshd -T, fail2ban-client get), no solo lo escrito en los archivos de configuración.
- Las instantáneas de VirtualBox (01-linea-base, 02-incidente-sin-defensa, 03-hardening-aplicado) permiten reproducir cada etapa.
- Los tiempos de los registros del servidor están en UTC; la hora de Argentina es UTC−3.

- Las pruebas de ataque se limitaron a fallos de autenticación manuales contra la propia VM. No se utilizaron herramientas de fuerza bruta ni se atacó ningún sistema ajeno.

Herramientas. VirtualBox 7.x, Ubuntu Server 24.04.5 LTS, OpenSSH, UFW, fail2ban 1.0.2, unattended-upgrades, PowerShell (cliente SSH en Windows 11) y Bash.

5. Resultados obtenidos

5.1 Comparación antes/después

Aspecto	Antes (línea base)	Después (hardening)
Acceso de root por SSH	PermitRootLogin without-password	PermitRootLogin no
Autenticación por contraseña	PasswordAuthentication yes	PasswordAuthentication no (solo clave pública)
Intentos por conexión	MaxAuthTries 6	MaxAuthTries 3
Reenvío X11	X11Forwarding yes	X11Forwarding no
Usuarios autorizados	No relevado en la línea base	AllowUsers admin1
Tiempo de gracia de login	No relevado en la línea base	LoginGraceTime 30
Firewall	UFW inactivo	UFW activo: entrada denegada, solo OpenSSH permitido
Respuesta a intentos repetidos	Ninguna (6 fallos sin consecuencias)	Bloqueo de la IP tras 3 fallos durante 10 minutos (fail2ban + UFW)
Actualizaciones pendientes	27	0
Actualizaciones de seguridad automáticas	Activas por defecto	Activas (verificadas)
Puerto 22 en escucha	0.0.0.0 y [::]	Sin cambios (protegido por UFW y autenticación por clave)

5.2 Evidencia del incidente

En la Etapa 2 se registraron 6 intentos fallidos, todos desde 192.168.56.1: 3 contra un usuario inexistente (prueba) y 3 contra admin1. El servidor no tomó ninguna acción (Figuras 12 a 14).

5.3 Efecto de las defensas

- **Contraseñas:** tras el hardening, el intento de entrar con contraseña es rechazado con Permission denied (publickey), incluso si la contraseña fuera correcta (Figura 31).

- **fail2ban:** al repetir los fallos, la IP fue bloqueada automáticamente (regla REJECT en UFW) y el cliente observó Connection timed out (Figuras 23 y 24).
- **Reproducibilidad:** el script hardening.sh, ejecutado sobre la VM restaurada a la línea base, logró el mismo estado final (Figuras 30 y 31).

5.4 Problemas encontrados y soluciones

Problema	Causa	Solución
ssh.service could not be found	La instalación automática no incluyó el servidor SSH	apt install openssh-server
Bad configuration option	El archivo se creó en ssh_config.d (cliente) en vez de sshd_config.d (servidor)	Mover el archivo al directorio correcto
fail2ban seguía con maxretry 5 y nftables	El servicio arrancó con valores por defecto antes de existir jail.local	systemctl restart fail2ban
Esperado Connection refused, observado Connection timed out	La regla de UFW descarta los paquetes en vez de responder	Se documentó el comportamiento real
Símbolos incorrectos al escribir en la consola de la VM	Distribución de teclado en inglés (EE. UU.)	Trabajar por SSH desde PowerShell y configurar el teclado latinoamericano

5.5 Limitaciones

- LoginGraceTime y AllowUsers no se midieron en la línea base, por lo que no se presentan valores "antes" para ellos.
- SSH sigue escuchando en todas las interfaces; una mejora posible sería restringirlo con ListenAddress o limitar el origen en UFW a la red 192.168.56.0/24.
- unattended-upgrades ya estaba activo por defecto y solo aplica actualizaciones de seguridad; no se modificó su alcance.
- fail2ban actúa *después* de los fallos y no protege contra el uso de una clave privada robada: por eso la clave tiene *passphrase*.
- Durante una prueba con *passphrase* se observó un mensaje Connection closed. Una hipótesis es que se alcanzó LoginGraceTime 30, pero no se confirmó y se menciona solo como hipótesis.
- El estudio se realizó en una sola VM y con un único origen de intentos; no pretende representar un ataque real distribuido.

6. Conclusiones

El trabajo mostró que un servidor con la instalación por defecto ya es utilizable, pero deja abiertas decisiones de seguridad que conviene tomar de forma explícita: acepta contraseñas, admite hasta 6 intentos por conexión, no tiene firewall activo y no reacciona ante intentos repetidos. Con pocas medidas combinadas (autenticación por clave, usuario único autorizado, firewall, bloqueo automático y actualizaciones) se redujo notablemente la superficie de ataque, y cada medida se pudo verificar con evidencia del propio sistema.

También dejó claro que ninguna medida es suficiente por sí sola. Cada una cubre una parte del problema (credenciales débiles, exposición de servicios, intentos reiterados, vulnerabilidades conocidas), por lo que el valor está en la defensa en profundidad. Del mismo modo, los errores encontrados (directorio equivocado, servicio iniciado antes de la configuración, diferencia entre lo escrito y lo efectivo) mostraron la importancia de verificar siempre los valores efectivos y de probar un acceso alternativo antes de cerrar una sesión remota.

Como aprendizaje sobre la materia, el caso permitió relacionar conceptos de sistemas operativos (procesos y servicios gestionados por systemd, usuarios y permisos, registros del sistema, red y firewall) con una situación concreta de administración.

Posibles mejoras y trabajo futuro: cambiar el puerto 22 puede ayudar a evitar muchos ataques que solo busquen el default, restringir ListenAddress o el origen permitido en UFW; incorporar auditd para auditar eventos del sistema; probar fail2ban con tiempos de bloqueo crecientes; y repetir el experimento con otro origen de ataque en una segunda VM.

7. Nota de transparencia

Para este trabajo fue utilizada inteligencia artificial generativa para ayudar en los siguientes puntos:

- Optimización de la redacción, correcciones de estilo y estructura académica del documento.
- Búsqueda del tema de estudio, proponiendo al alumno diferentes temas que se ajustaran a lo requerido por la cátedra.

La responsabilidad intelectual sobre la veracidad y coherencia conceptual de la entrega recae íntegramente en el autor.

8. Bibliografía

Fail2ban Project. (s. f.). *Fail2ban* [Software y documentación]. GitHub. https://github.com/fail2ban/fail2ban

Canonical Ltd. (s. f.). *UFW – Uncomplicated Firewall*. Ubuntu Community Help Wiki. https://help.ubuntu.com/community/UFW

Canonical Ltd. (s. f.). *Automatic security updates*. Ubuntu Community Help Wiki. https://help.ubuntu.com/community/AutomaticSecurityUpdates

Canonical Ltd. (s. f.). *Ubuntu Server documentation*. https://ubuntu.com/server/docs

OpenBSD Project. (s. f.). *sshd_config(5) – OpenSSH daemon configuration file*. https://man.openbsd.org/sshd_config

Oracle. (s. f.). *Oracle VM VirtualBox user manual*. https://www.virtualbox.org/manual/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

9. Anexos

Anexo A. Script hardening.sh

También se entrega como archivo en la carpeta codigo/.

```
#!/usr/bin/env bash

# hardening.sh - Hardening basico de acceso SSH en Ubuntu Server 24.04

# Uso: sudo bash hardening.sh [usuario] # por defecto: admin1

#   IMPORTANTE:   el   usuario   debe   tener   su   clave   publica   en

~/.ssh/authorized_keys.

set -euo pipefail


ADMIN_USER="${1:-admin1}"
```

```
SSHD_DROPIN="/etc/ssh/sshd_config.d/00-hardening.conf"
AUTH_KEYS="/home/${ADMIN_USER}/.ssh/authorized_keys"

info() { printf '\n\033[1;34m[+] %s\033[0m\n' "$*"; }
fail() { printf '\n\033[1;31m[!] %s\033[0m\n' "$*" >&2; exit 1; }

# 0. Comprobaciones previas
[[ $EUID -eq 0 ]] || fail "Ejecutar con sudo: sudo bash $0 [usuario]"
id "$ADMIN_USER" >/dev/null 2>&1 || fail "El usuario '$ADMIN_USER' no
existe."
[[ -s "$AUTH_KEYS" ]] || fail "No hay clave publica en $AUTH_KEYS. Cargala
antes de continuar."

# 1. SSH endurecido (en sshd_config.d gana el PRIMER valor: por eso 00-)
info "Configurando SSH ($SSHD_DROPIN)"
apt-get install -y openssh-server >/dev/null
cat > "$SSHD_DROPIN" <<EOF
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowUsers ${ADMIN_USER}
EOF
```

```
# Validar antes de aplicar; si es invalida, se revierte.
if ! sshd -t; then
rm -f "$SSHD_DROPIN"
fail "La configuracion de SSH es invalida; se revirtio el cambio."
fi
systemctl restart ssh

# 2. Firewall UFW (permitir SSH ANTES de activar)
info "Configurando firewall (UFW)"
apt-get install -y ufw >/dev/null
ufw default deny incoming
ufw default allow outgoing
ufw allow OpenSSH
ufw --force enable

# 3. fail2ban
info "Configurando fail2ban"
apt-get install -y fail2ban >/dev/null
cat > /etc/fail2ban/jail.local <<'EOF'
[DEFAULT]
bantime = 10m
findtime = 10m
maxretry = 3
banaction = ufw
```

```
[sshd]
enabled = true
backend = systemd
EOF
# El servicio arranca con la config por defecto: hay que REINICIARLO.
systemctl enable fail2ban
systemctl restart fail2ban

# 4. Actualizaciones automaticas de seguridad
info "Verificando actualizaciones automaticas"
apt-get install -y unattended-upgrades >/dev/null
dpkg-reconfigure -f noninteractive unattended-upgrades

# Verificacion: valores EFECTIVOS

      info "Verificacion"
echo "--- sshd -T ---"
sshd               -T               |               grep               -Ei
'permitrootlogin|passwordauthentication|pubkeyauthentication|maxauthtries|x
11forwarding|allowusers|logingracetime'
echo "--- ufw ---"
ufw status verbose
echo "--- fail2ban (valores efectivos) ---"
echo "maxretry: $(fail2ban-client get sshd maxretry)"
```

```
fail2ban-client get sshd actions

fail2ban-client status sshd


info "Listo. Probar en OTRA terminal que el acceso por clave funciona antes

de cerrar esta sesion."
```

(La versión entregada en codigo/hardening.sh conserva los comentarios completos del encabezado.)

Anexo B. Comandos de verificación utilizados

sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthentication|pubkeyauthentication|maxauthtries|x11forwarding|allowusers|logingracetime'
sudo ufw status verbose
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip 192.168.56.1
sudo journalctl -u ssh --no-pager | grep -c "Failed password"
systemctl list-timers apt-daily*
sudo unattended-upgrade --dry-run -d
apt list --upgradable
diff ~/evidencia/antes/ssh_config.txt ~/evidencia/despues/ssh_config.txt

Anexo C. Enlaces y entregables

- Video de presentación: https://www.youtube.com/watch?v=5O4gtPTf2lI
- Repositorio / carpeta digital: https://github.com/valenzuelagustavo/tpi-hardening-ssh